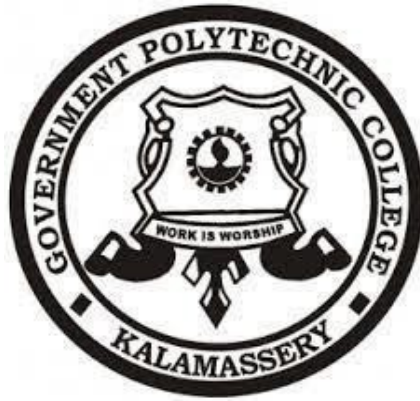


**GOVERNMENT POLYTECHNIC  
COLLEGE  
KALAMASSERY**



**DEPARTMENT OF COMPUTER ENGINEERING  
2025 – 2026**

In partial fulfillment of the requirements for the award of Diploma in  
Computer Engineering

**Project report on  
STUDENT COMPLAINT REGISTRATION SYSTEM**

Submitted By

|                    |            |
|--------------------|------------|
| ADITHYAN T P       | 2401132035 |
| ASWIN R            | 2401132046 |
| M M MUHAMMED RIYAS | 2401132058 |

**GOVERNMENT POLYTECHNIC COLLEGE  
KALAMASSERY**



**DEPARTMENT OF COMPUTER ENGINEERING  
2025 – 2026**

**CERTIFICATE**

This is to certify that the project report entitled "**STUDENT COMPLAINT REGISTRATION SYSTEM**" is submitted by **ADITHYAN T P, ASWIN R, M M MUHAMMED RIYAS** of Fourth Semester Diploma in Computer Engineering for their partial fulfilment of the requirement for the award of Diploma in Computer Engineering under the Department of Technical Education Govt. of Kerala during the academic year 2025-2026 at Govt. POLYTECHNIC COLLEGE, KALAMASSERY.

**INTERNAL EXAMINER**

**STAFF IN CHARGE**

**EXTERNAL EXAMINER**

**HEAD OF SECTION**

## **ACKNOWLEDGEMENT**

First and foremost, we give thanks to Almighty God who has given us the inner strength, resources and ability to complete the project successfully.

We would like to take this opportunity to express our profound gratitude and deep regards to our guide and Head of the Department **Mrs. SEENA** , for his exemplary guidance, monitoring and constant encouragement throughout the course of this project.

We express our sincere gratitude to our class tutor **Mrs. MARY MURPHY** for her cordial support valuable information and guidance which made this project a great success.

We would like to express our special thanks to our project guide **Mrs. CHITRA** and other faculty members of the department for encouragement and their cordial support, valuable information and guidance which made this project a great success.

We would like to thank all of our teachers and staffs of Department of Computer Engineering.

At last, thanks go to our Friends who have helped us to make our work more organized and well stacked till the end.

## **INDEX**

| SL NO | CONTENTS                        | PAGE NO |
|-------|---------------------------------|---------|
| 1     | INTRODUCTION                    | 5       |
| 2     | SCOPE OF THE PROJECT            | 6       |
| 3     | EXISTING SYSTEM                 | 8       |
| 4     | PROPOSED PROJECT                | 11      |
| 5     | FEASIBILITY STUDY               | 13      |
| 6     | SYSTEM SPECIFICATION            | 15      |
| 7     | COST AND BENEFIT ANALYSIS       | 17      |
| 8     | SYSTEM DESIGN                   | 18      |
| 9     | APPENDIX (CODING)               | 22      |
| 10    | SYSTEM TESTING & IMPLEMENTATION | 40      |
| 11    | SYSTEM MAINTENANCE              | 41      |
| 12    | FUTURE SCOPE                    | 43      |
| 13    | CONCLUSION                      | 45      |
| 14    | REFERENCES                      | 46      |

# **STUDENT COMPLAINT REGISTRATION SYSTEM**

## **1. INTRODUCTION**

Managing student grievances effectively is one of the most important aspects of a well-functioning educational institution. At Government Polytechnic College, Kalamassery, the existing process for handling student complaints is primarily manual — students write complaints on paper or approach faculty verbally, making it difficult to ensure that grievances are formally recorded, properly escalated, and transparently resolved. This lack of a structured mechanism leads to complaints being lost, unacknowledged, or resolved without any record of the actions taken.

This project presents the Student Complaint Registration System (SCRS) — a web-based application built using the Python Django framework. The system provides a formal, digital, and hierarchical platform through which students can register complaints online and track their progress in real time as they move through a structured multi-level approval workflow involving the Class Tutor, Head of Department (HOD), and Principal.

The system is designed for Government Polytechnic College, Kalamassery, which currently offers Diploma programmes across six engineering departments: Computer Engineering (CT), Electronics and Communication Engineering (ECE), Mechanical Engineering (MECH), Civil Engineering (CIVIL), Electrical and Electronics Engineering (EEE), and Automobile Engineering (AUTO). Students in all six departments across all six semesters can use the system.

Four distinct user roles are supported by the system. The Student role allows learners to create, edit, submit, and track complaints. The Class Tutor role allows semester-level faculty to review and act on complaints from their class. The Head of Department (HOD) role allows department-level authorities to review and escalate complaints. Finally, the Principal role provides the highest level of oversight, approving department-endorsed complaints and marking them as resolved. Each role has a dedicated, purpose-built dashboard.

A key design consideration was the invitation-based and request-based registration system. Students may either be pre-registered by an authority (Class Tutor, HOD, or Principal), use an invitation code issued by an authority to self-register immediately, or submit a self-registration request that is reviewed and approved by the department HOD. This dual-path registration model balances institutional control with practical accessibility.

The system also includes a complaint category structure (Infrastructure, Academics, Hostel, Canteen, Transport, Library, Sports, and Other), mandatory review comments at every approval stage, in-app notifications for every status change, file attachment support, profile picture uploads, and a dark mode preference. An administrative semester increment tool allows the Principal to promote all students to the next semester at the end of each academic term.

By replacing ad-hoc, paper-based grievance handling with a formal digital workflow, the Student Complaint Registration System aims to promote accountability, transparency, and student welfare at Government Polytechnic College, Kalamassery.

## **2. SCOPE OF THE PROJECT**

The Student Complaint Registration System is designed to address the complete grievance management lifecycle within Government Polytechnic College, Kalamassery. The scope encompasses user management, complaint registration, multi-level approval workflow, notifications, and administrative tools.

### **2.1 User Management**

The system manages four roles in a hierarchical structure. The Principal sits at the top and can create accounts for HODs, Class Tutors, and Students across all departments. The HOD manages users within their own department, including Class Tutors and Students. The Class Tutor manages Students within their specific department and semester. Students can be created by an authority or can self-register using an invitation code.

A dual-role mechanism is supported: an HOD can optionally also act as a Class Tutor for one specific semester within their department. This is useful for smaller departments where a dedicated Class Tutor may not be available for every semester. The `is_also_class_tutor` and `tutor_semester` fields on the User model enable this functionality.

### **2.2 Complaint Lifecycle Management**

Students can create complaint drafts, review them, and submit them when ready. The draft state allows students to compose and edit their complaint before it enters the formal workflow, preventing accidental or premature submissions. Once submitted, the complaint moves through the following status chain:

- DRAFT: Complaint created but not yet submitted.
- PENDING: Complaint submitted by the student; awaiting Class Tutor review.
- APPROVED\_BY\_TUTOR: Class Tutor has approved the complaint; awaiting HOD review.
- APPROVED\_BY\_HOD: HOD has approved the complaint; awaiting Principal review.
- APPROVED\_BY\_PRINCIPAL: Principal has approved the complaint; to be resolved.
- REJECTED: Complaint rejected by any authority at any stage.
- RESOLVED: Principal has marked the complaint as resolved.

### **2.3 Complaint Categories**

The system supports eight predefined complaint categories to ensure consistent classification and routing: Infrastructure, Academics, Hostel, Canteen, Transport, Library, Sports, and Other. Students may also provide a custom category name when selecting 'Other'. Complaints are grouped by category on the admin dashboards for efficient management.

### **2.4 Notifications and Comments**

Every status change in a complaint triggers an in-app notification for the student. Notifications are stored in the `ComplaintStatusNotification` model and displayed on the student dashboard. Authorities must provide a mandatory comment when approving or rejecting a complaint, providing the student with documented feedback. Comments are stored in the `ComplaintComment` model and are visible on the complaint detail page.

## **2.5 Registration Mechanisms**

Two registration paths are available for students. In the invite-based path, an authority creates an Invite record linked to the student's ID. When the student enters this ID during registration, the system pre-fills their details and creates the account immediately upon submission. In the request-based path, the student fills in their own details and submits a RegistrationRequest, which is reviewed by the HOD of the selected department. The HOD can approve or reject the request from their dashboard.

## **2.6 Semester Increment Tool**

The Principal has access to a semester increment utility that promotes all students to the next semester at the end of an academic term. Students currently in Semester 6 are automatically removed from the system (as they have graduated), while all other students are incremented by one semester. This ensures that the system always reflects the current academic state of the student body.

## **2.7 Out of Scope**

The following features are outside the current scope of the project: real-time email or SMS notifications, integration with external banking or payment systems, a mobile application, multi-institution support, and AI-based complaint analysis. These are identified as future enhancements.

### **3. EXISTING SYSTEM**

Before evaluating the proposed solution, it is important to understand the current state of student grievance management in polytechnic colleges. This section analyses the existing processes, tools, and their inherent limitations.

#### **3.1 Manual Paper-Based Complaint Submission**

The most prevalent method for handling student complaints in Government Polytechnic College is paper-based. A student with a grievance is expected to write a complaint letter, fill in a physical form, or speak directly with their Class Tutor. The Class Tutor either addresses the issue personally or escalates it verbally or in writing to the HOD. The HOD, if the issue warrants it, further escalates to the Principal's office.

This manual chain suffers from several well-documented problems. Physical documents can be misplaced or forgotten in busy administrative environments. There is no consistent record-keeping of how many complaints were received, how long they took to resolve, or what actions were taken. Students have no visibility into the status of their complaint once it leaves their hands. There is also no accountability mechanism — if a Class Tutor fails to escalate a legitimate complaint, the student has no recourse.

#### **3.2 Verbal Communication**

Many students raise concerns verbally during class, in corridors, or during office hours. While this is the most natural form of communication, it is entirely undocumented. A verbally expressed grievance relies entirely on the goodwill and memory of the faculty member who receives it. There is no tracking, no escalation path, and no formal acknowledgement to the student. In cases where the grievance involves a sensitive matter (such as a complaint against a specific faculty member), the student may be reluctant to raise the issue verbally with someone close to the subject of the complaint.

#### **3.3 Email-Based Complaints**

Some students and faculty members use institutional email for complaint communication. While this provides a record, emails are unstructured — they can be addressed to any recipient, can bypass the proper hierarchy, and are not accessible to all relevant parties simultaneously. There is no workflow enforcement: an email can sit in a busy inbox without being acted upon. The recipient cannot easily mark it with a status, and the sender has no way to know whether the email has been read, forwarded, or actioned.

#### **3.4 Review of Comparable Existing Solutions**

Several general-purpose and academic complaint management tools exist, but none is well-suited to the specific requirements of a polytechnic college hierarchy.

##### **3.4.1 Microsoft Excel / Google Sheets**

Some administrative staff maintain spreadsheets to log student complaints. While this provides a basic digital record, it requires manual updates, offers no workflow automation, cannot enforce a multi-level approval chain, does not generate notifications, and is not accessible to students. A student cannot view the status of their own complaint in a shared spreadsheet.

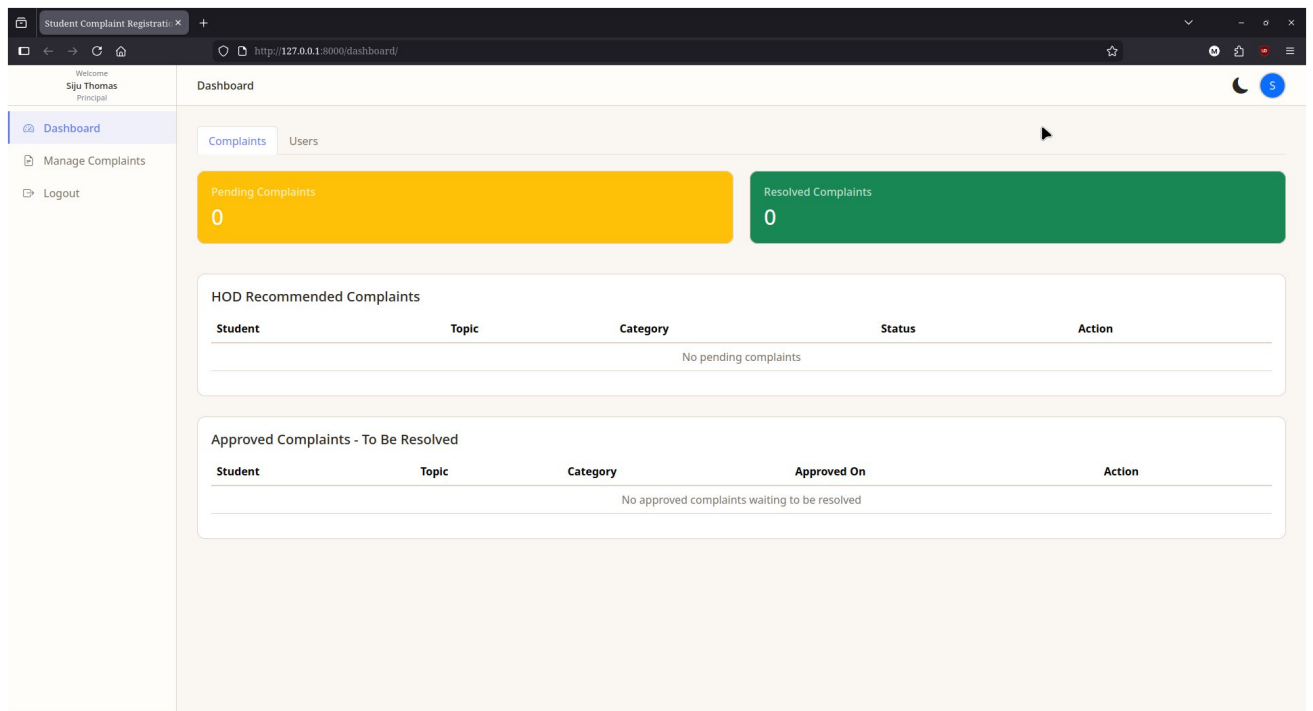


### **3.5 Summary of Limitations**

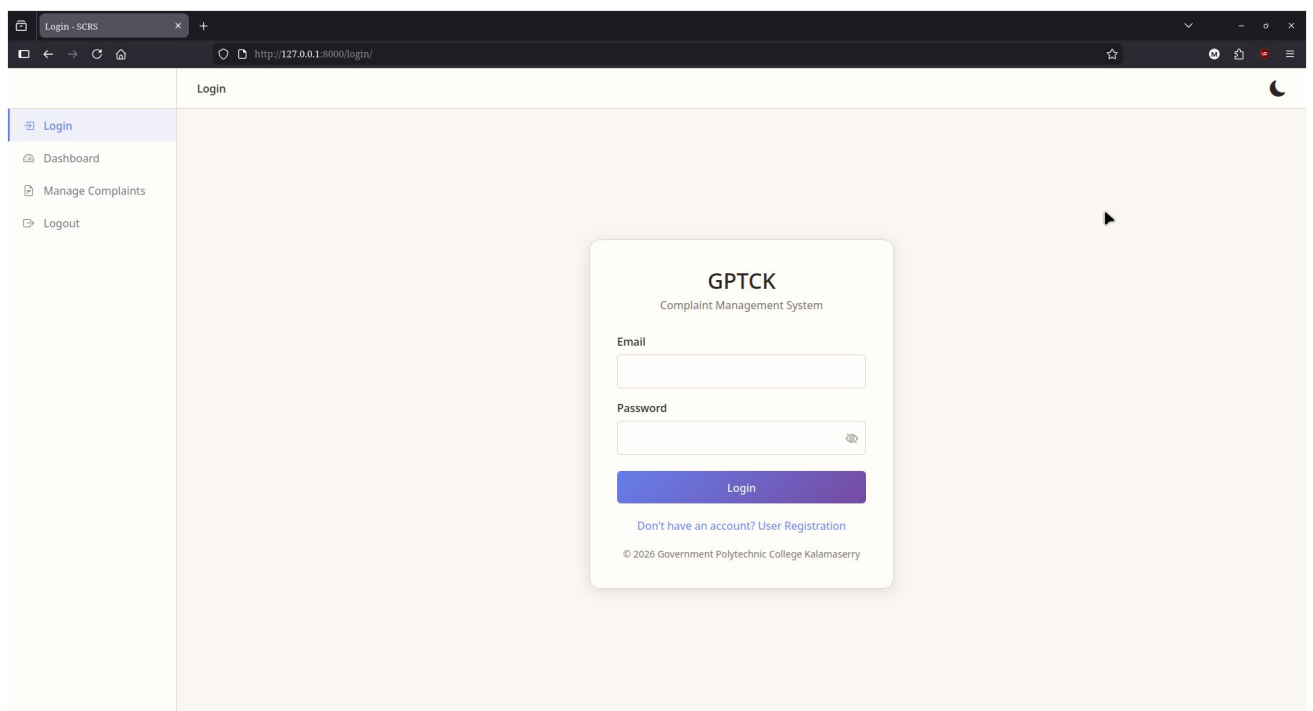
| Limitation                   | Existing Methods   | Impact                                 |
|------------------------------|--------------------|----------------------------------------|
| No tracking mechanism        | Paper, verbal      | Complaints lost or forgotten           |
| No role-based visibility     | All methods        | Sensitive data exposed inappropriately |
| No status updates to student | All methods        | Student unaware of complaint progress  |
| No audit trail               | Paper, verbal      | Accountability cannot be enforced      |
| No escalation enforcement    | All methods        | Complaints stall at lower levels       |
| No structured workflow       | Email, spreadsheet | Bypasses proper authority chain        |
| Access control absent        | Email, spreadsheet | Unauthorised access possible           |

The Student Complaint Registration System proposed in this project directly addresses each of these limitations through structured digital workflow, role-based access control, real-time status tracking, mandatory comments, and automated notifications.

# Student Complaint Registration System



## SCRS



## **4. PROPOSED PROJECT**

The Student Complaint Registration System (SCRS) is a web-based application developed using the Python Django framework. It provides a formal, transparent, and auditable grievance management platform for Government Polytechnic College, Kalamassery. The system replaces informal and manual complaint handling with a structured digital workflow that enforces accountability at every level of the academic hierarchy.

### **4.1 Core Objectives**

- To provide students with a formal and accessible channel to register complaints digitally.
- To enforce a structured three-level approval workflow: Class Tutor, HOD, Principal.
- To ensure role-based access control so that each authority sees only the complaints within their jurisdiction.
- To provide real-time status notifications to students at every stage of the complaint lifecycle.
- To maintain a complete and auditable history of every complaint, including comments, timestamps, and reviewer identities.
- To support multiple registration pathways for students while maintaining institutional control.
- To provide the Principal with administrative tools for user management and semester advancement.

### **4.2 Key Features**

#### **4.2.1 Role-Based Dashboards**

Each of the four user roles is presented with a dedicated dashboard tailored to their responsibilities. The student dashboard shows complaint counts by status and lists active, resolved, and rejected complaints. The Class Tutor dashboard shows complaints from their assigned semester and department that require review. The HOD dashboard shows complaints from their department that have been approved by the Class Tutor. The Principal dashboard provides a college-wide view of HOD-approved complaints.

#### **4.2.2 Multi-Level Complaint Workflow**

The complaint workflow follows a linear approval chain. A student creates and submits a complaint. The Class Tutor reviews it and either approves (forwarding it to the HOD) or rejects it with a mandatory comment. The HOD reviews Tutor-approved complaints and either approves (forwarding to the Principal) or rejects them. The Principal reviews HOD-approved complaints, approves them, and can subsequently mark them as resolved.

#### **4.2.3 Draft and Submit Mechanism**

Students can save complaints as drafts before formal submission. A draft complaint is only visible to the student and does not enter the workflow. The student can edit, update, or delete drafts at any time. When ready, the student submits the draft, changing the status to PENDING and making it visible to the Class Tutor.

#### **4.2.4 Invitation and Registration System**

Two registration paths are supported. The invite path allows an authority (Class Tutor, HOD, or Principal) to pre-register a student by creating an Invite record. The student uses their invitation ID during self-registration to have their account created immediately. The request path allows students without an invitation to submit a RegistrationRequest; the HOD reviews and approves or rejects the request.

#### **4.2.5 Temporary Password Flow**

When an authority creates an account for an HOD or Class Tutor directly from the dashboard, a temporary password is set and the `must_change_password` flag is set to `True`. On the user's first login, they are prompted to change this password before accessing any system functionality. This ensures that no account remains accessible with a known temporary credential.

#### **4.2.6 Semester Increment Tool**

The Principal can trigger a semester increment operation from the dashboard. This operation removes all Semester 6 student accounts (as those students have completed the diploma programme) and increments the semester of all remaining active students by one. This keeps the user database aligned with the current academic calendar.

#### **4.2.7 File Attachments and Profile Pictures**

Students can optionally attach a supporting document or image when submitting a complaint. The attachment is stored in the `media/complaints/` directory and can be viewed by any authority reviewing the complaint. Users can also upload a profile picture from the settings page, stored in `media/profiles/`. Pillow is used for image handling.

### **4.3 Technology Stack**

| Component         | Technology                                |
|-------------------|-------------------------------------------|
| Backend Framework | Python Django 4.2.x                       |
| Database          | SQLite (development)                      |
| Frontend          | HTML5, CSS3, Bootstrap 5, Bootstrap Icons |
| Authentication    | Django AbstractBaseUser with email login  |
| Image Processing  | Pillow 10.0.0+                            |
| Version Control   | Git                                       |

## **5. FEASIBILITY STUDY**

A comprehensive feasibility study was conducted before commencing development to evaluate the practicality, viability, and sustainability of the Student Complaint Registration System. Five dimensions of feasibility were analysed.

### **5.1 Technical Feasibility**

The project relies entirely on established, open-source technologies that are well-documented and widely deployed in production environments worldwide. Python Django 4.2 is a mature, battle-tested framework with a large community and extensive documentation. SQLite, used as the development database, requires no separate server installation and is bundled with Python. Bootstrap 5 provides a mature, responsive CSS framework.

The custom user model (AbstractBaseUser) required to support email-based login with multiple roles is a standard Django extension pattern with no non-standard dependencies. The complaint workflow logic is implemented entirely as Django ORM queries and view functions, requiring no external APIs, machine learning libraries, or proprietary tools.

- All required Python packages are listed in requirements.txt and installable via `pip install -r requirements.txt`.
- The project was developed and tested on Python 3.12, which is widely available on all major operating systems.
- The system can be deployed on any Linux-based server or cloud platform supporting Python (e.g., DigitalOcean, Render, PythonAnywhere).
- For production deployment, the SQLite database can be replaced with PostgreSQL with minimal code changes, as Django's ORM abstracts the database layer.

### **5.2 Economic Feasibility**

The project was developed at near-zero software cost. All tools and frameworks used are open-source and freely licensed. Development was carried out on existing student laptops. A breakdown of costs is provided in Chapter 7.

The system replaces paper-based processes that incur recurring stationery, printing, and administrative costs. By digitising grievance management, the college can reduce these costs and redirect administrative time to more productive activities. The system also prevents disputes arising from lost or unacknowledged complaints, which may otherwise escalate into costly institutional conflicts.

### **5.3 Operational Feasibility**

The user interface is built on Bootstrap 5, which provides a clean, mobile-responsive design without requiring any custom CSS framework expertise. Role-specific dashboards reduce cognitive load — each user sees only the information and actions relevant to their role.

The login page doubles as the student registration page. Students can register without visiting any administrative office. The HOD dashboard provides a dedicated section for reviewing registration requests. This self-service model reduces the operational burden on administrative staff.

The temporary-password flow ensures that all authority-created accounts are secured on first login. The dark mode preference, stored per user in the database, improves usability in low-light environments such as computer labs at night.

- The system requires no training for students beyond basic web browsing skills.
- Faculty and administrators require a brief orientation to their role-specific dashboards.
- The `setup_principal.py` script allows the Principal account to be created or reset with a single terminal command, minimising IT support requirements.

#### **5.4 Legal and Ethical Feasibility**

Student and staff data stored in the system is protected by Django's built-in security mechanisms. Passwords are hashed using PBKDF2 with SHA-256 and are never stored in plaintext. CSRF protection is applied to all form submissions. Session-based authentication ensures that users are properly authenticated before accessing any system view.

Role-based access control ensures that complaint data is visible only to parties with legitimate authority over the complaint. A student cannot view another student's complaints. A Class Tutor cannot view complaints from a different department or semester. An HOD cannot view complaints that have not been escalated to their level. These controls ensure that the system complies with basic privacy expectations.

- No sensitive financial data is stored. The system does not process payments or link to banking systems.
- The system provides unbiased grievance handling — all complaints of the same category are subject to the same workflow, regardless of the student's identity.
- The mandatory comment requirement at every approval or rejection stage ensures that decisions are documented and defensible.

#### **5.5 Schedule Feasibility**

- The development timeline is reasonable given the use of Django's built-in functionalities.
- A well-structured roadmap ensures timely completion and deployment.

Django's built-in ORM, form handling, and admin interface significantly reduced development time at each phase. Bootstrap 5's component library accelerated frontend development, allowing the team to focus on backend logic rather than CSS styling.

## 6. SYSTEM SPECIFICATIONS

### 6.1 Hardware Specifications

| Component | Minimum Requirement                                          |
|-----------|--------------------------------------------------------------|
| Processor | Intel Core i3 (1.8 GHz or higher)                            |
| RAM       | 4 GB minimum; 8 GB recommended for development               |
| Storage   | 128 GB SSD (256 GB recommended for media storage)            |
| Network   | LAN or Wi-Fi for local access; Internet for cloud deployment |
| Display   | 1024×768 resolution minimum for UI rendering                 |

### 6.2 Software Specifications

| Software Component    | Version / Details                          |
|-----------------------|--------------------------------------------|
| Operating System      | Windows 10/11, macOS 12+, or Ubuntu 20.04+ |
| Python Runtime        | Python 3.10 or higher                      |
| Web Framework         | Django 4.2.x                               |
| Database              | SQLite 3 (bundled with Python)             |
| Frontend Framework    | Bootstrap 5.3.2                            |
| Icon Library          | Bootstrap Icons 1.11.1                     |
| Image Processing      | Pillow 10.0.0+                             |
| Web Server            | Django built-in development server         |
| Version Control       | Git 2.x                                    |
| Browser Compatibility | Chrome 110+, Firefox 110+, Safari 16+      |

### 6.3 Python Dependencies (requirements.txt)

```
Django>=4.2,<5.0
Pillow>=10.0.0
```

### 6.4 Project Structure

```
scrs/
├── accounts/           # User management app
│   ├── models.py      # User, Invite, RegistrationRequest, AdmissionNumber
│   ├── views.py       # Login, register, dashboard, user CRUD, semester increment
│   ├── urls.py        # URL routing for accounts
│   └── admin.py       # Django admin configuration
├── complaints/       # Complaint management app
│   ├── models.py      # Complaint, ComplaintComment, ComplaintStatusNotification
│   ├── views.py       # Complaint CRUD, workflow actions, notifications
│   ├── urls.py        # URL routing for complaints
├── config/           # Django project configuration
└── settings.py       # Project settings
```

```

├── urls.py           # Root URL configuration
├── wsgi.py           # WSGI application entry point
├── asgi.py           # ASGI application entry point
├── templates/        # HTML templates
│   ├── accounts/     # Login, settings, semester increment templates
│   ├── complaints/   # Dashboard, complaint management templates
│   └── shared/        # Base template with sidebar navigation
├── static/           # Static files (CSS)
│   └── css/
│       ├── base.css  # Main stylesheet
│       └── login.css # Login page stylesheet
├── media/            # User-uploaded files (created at runtime)
│   ├── profiles/     # Profile pictures
│   └── complaints/   # Complaint attachments
├── manage.py         # Django management script
├── setup_principal.py # Script to create/reset Principal account
└── requirements.txt  # Python dependencies

```

## 6.5 URL Structure

| URL Pattern                | Purpose                                     |
|----------------------------|---------------------------------------------|
| /                          | Redirects to /login/                        |
| /login/                    | Login and student registration page         |
| /logout/                   | Logs out the current user                   |
| /dashboard/                | Role-specific dashboard                     |
| /settings/                 | User profile and password settings          |
| /users/create/             | Create a new user account (admin only)      |
| /users/<id>/edit/          | Edit a user account (admin only)            |
| /users/<id>/delete/        | Delete a user account (admin only)          |
| /users/semester-increment/ | Semester increment tool (Principal only)    |
| /complaints/manage/        | Complaint list (role-scoped)                |
| /complaints/view/<id>/     | View complaint details and comments         |
| /complaints/edit/<id>/     | Edit a draft complaint (student only)       |
| /complaints/action/<id>/   | Approve or reject a complaint (admin only)  |
| /complaints/resolve/<id>/  | Mark complaint as resolved (Principal only) |



## **7. COST AND BENEFIT ANALYSIS**

### **1. Hardware Cost**

- Development requires a mid-range laptop or cloud server.

### **2. Software Cost**

- Open-source software minimizes costs.

### **3. Personal Cost**

- Developer and maintenance personnel costs.

### **4. Operating Cost and Supply Cost**

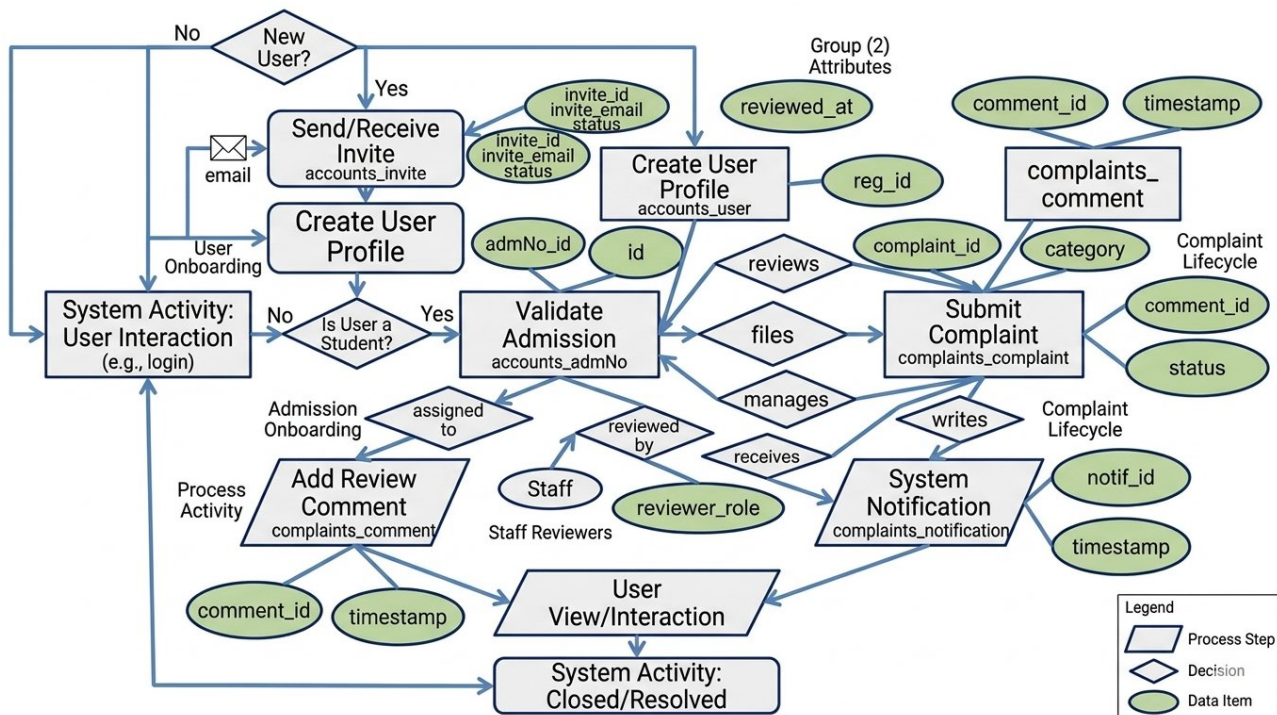
- Server hosting and API integration fees (if applicable).

## 8. SYSTEM DESIGN

### 8.1 Data Flow Diagram – Level 0 (Context Diagram)

At the highest level of abstraction, the system interacts with four external entities: Student, Class Tutor, HOD, and Principal. The system receives complaint data and user management instructions from these entities and returns status updates, notifications, and approval outcomes.

- Student → System: Registration request, complaint creation, complaint submission, draft edits, file attachments.
- Class Tutor → System: Complaint approval or rejection with comment, user account management within class.
- HOD → System: Complaint approval or rejection, department user management, registration request review.
- Principal → System: College-wide complaint oversight, resolution, user management, semester increment.
- System → Student: Complaint status notifications, complaint detail view, draft management.
- System → All Roles: Role-scoped dashboard, complaint lists, audit trail.



### 8.2 Data Flow Diagram – Level 1

At Level 1, the system is decomposed into six major processes:

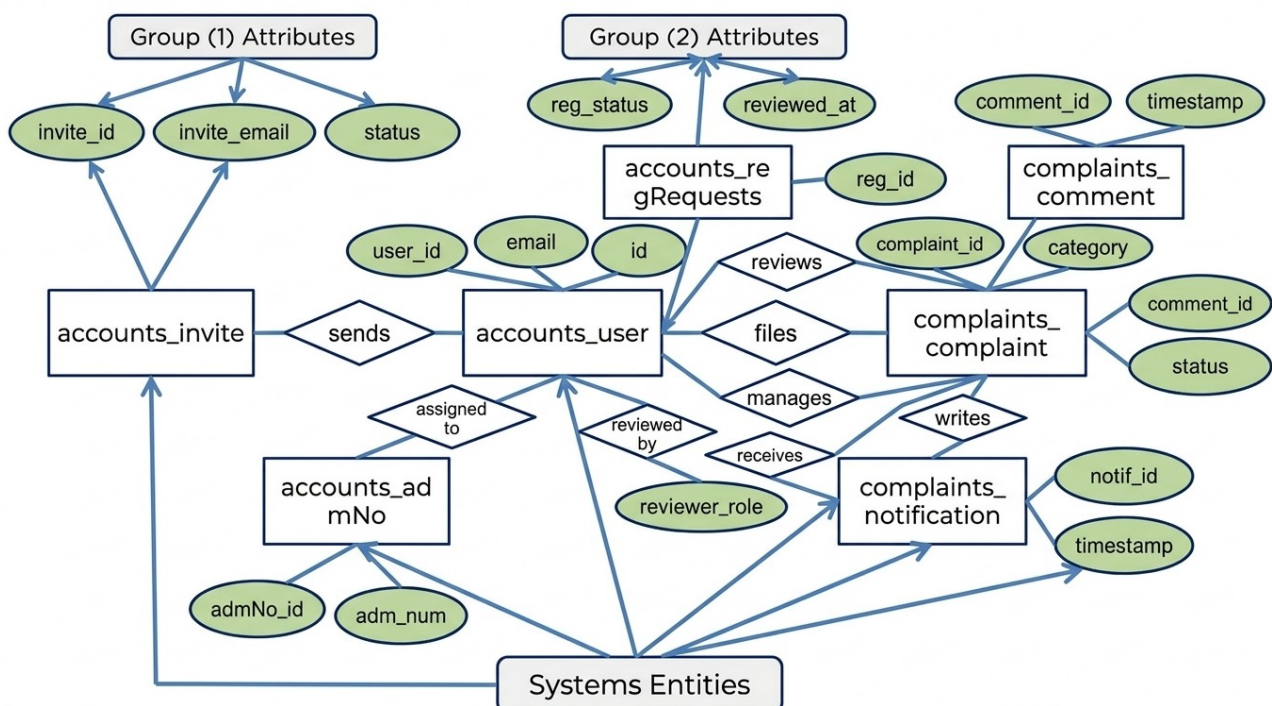
1. **Authentication and Session Management:** Validates user credentials, manages sessions, enforces must\_change\_password redirect.
2. **User and Registration Management:** Handles account creation by authorities, invite generation, student self-registration, and HOD review of registration requests.
3. **Complaint Creation and Submission:** Student creates draft, edits content, attaches file, and submits complaint to the workflow.

4. Complaint Review and Approval Workflow: Role-based view filtering, approval/rejection with mandatory comment, status transition logic.
5. Notification Generation: On every status change, creates ComplaintStatusNotification for the student.
6. Administrative Tools: Semester increment operation, profile and password management.

### 8.3 Entity-Relationship (ER) Diagram Description

The system's data model contains seven entities with the following relationships:

| Entity                      | Key Relationships                                                                                                                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User                        | Central entity. One User can create many Invites (created_by). One User can be referenced by many Complaints (student). One User can review many Complaints (reviewed_by_tutor / hod / principal). One User can have many Notifications. |
| Invite                      | Created by one User. May be used by one User (used_by). Tracks whether the invitation has been consumed.                                                                                                                                 |
| AdmissionNumber             | Standalone entity storing valid admission numbers for verification. Linked to User when consumed.                                                                                                                                        |
| RegistrationRequest         | Submitted by an anonymous registrant. Reviewed by one User (HOD). Stores hashed password for deferred account creation.                                                                                                                  |
| Complaint                   | Belongs to one User (student). Has status transitions. Has foreign keys to three reviewing Users (tutor, HOD, principal). Has many Comments. Has many Notifications.                                                                     |
| ComplaintComment            | Belongs to one Complaint. Written by one User. Marked as rejection comment when associated with a rejection action.                                                                                                                      |
| ComplaintStatusNotification | Belongs to one Complaint. Addressed to one User (student). Tracks read status.                                                                                                                                                           |



## 8.4 Database Schema

| Table                   | PK / FK Fields                     | Key Fields (1)                   | Key Fields (2)                            | Notes                     |
|-------------------------|------------------------------------|----------------------------------|-------------------------------------------|---------------------------|
| accounts_user           | id (PK)                            | email (unique), user_id (unique) | role, department, semester                | Custom AbstractBaseUser   |
| accounts_invite         | id (PK), created_by (FK→User)      | user_id, email, full_name        | is_used, used_by (FK→User)                | Invite-based registration |
| accounts_regRequest     | id (PK), reviewed_by (FK→User)     | user_id, email, status           | password_hash, department                 | HOD-reviewed requests     |
| accounts_admNo          | id (PK), used_by (FK→User)         | admission_number (unique)        | is_used, used_at                          | Admission validation      |
| complaints_complaint    | id (PK), student (FK→User)         | topic, category, status          | reviewed_by_tutor/hod/principal (FK→User) | Core complaint record     |
| complaints_comment      | id (PK), complaint (FK), user (FK) | comment, created_at              | is_rejection_comment                      | Approval/rejection notes  |
| complaints_notification | id (PK), complaint (FK), user (FK) | message, is_read                 | created_at                                | Student alerts            |

## 8.5 Complaint Status Transition Diagram

The following table describes every possible status transition in the complaint workflow, including the actor responsible for each transition and the condition under which it occurs:

| From Status       | To Status         | Actor       | Condition                                  |
|-------------------|-------------------|-------------|--------------------------------------------|
| —                 | DRAFT             | Student     | Student creates new complaint              |
| DRAFT             | DRAFT             | Student     | Student edits and saves without submitting |
| DRAFT             | PENDING           | Student     | Student clicks Submit                      |
| DRAFT             | (deleted)         | Student     | Student deletes draft                      |
| PENDING           | APPROVED_BY_TUTOR | Class Tutor | Tutor approves with comment                |
| PENDING           | REJECTED          | Class Tutor | Tutor rejects with comment                 |
| APPROVED_BY_TUTOR | APPROVED_BY_HOD   | HOD         | HOD approves                               |

|                       |                       |           |                                 |
|-----------------------|-----------------------|-----------|---------------------------------|
|                       |                       |           | with comment                    |
| APPROVED_BY_TUTOR     | REJECTED              | HOD       | HOD rejects with comment        |
| APPROVED_BY_HOD       | APPROVED_BY_PRINCIPAL | Principal | Principal approves with comment |
| APPROVED_BY_HOD       | REJECTED              | Principal | Principal rejects with comment  |
| APPROVED_BY_PRINCIPAL | RESOLVED              | Principal | Principal confirms resolution   |

## 9. CODING

### 9.1 accounts/models.py

```
from django.contrib.auth.models import (
    AbstractBaseUser, BaseUserManager, PermissionsMixin)
from django.db import models
from django.utils import timezone

class UserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError('Email is required')
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        extra_fields.setdefault('role', 'PRINCIPAL')
        return self.create_user(email, password, **extra_fields)

class User(AbstractBaseUser, PermissionsMixin):
    ROLE_CHOICES = [
        ('PRINCIPAL', 'Principal'), ('HOD', 'HOD'),
        ('CLASS_TUTOR', 'Class Tutor'), ('STUDENT', 'Student'),
    ]
    DEPARTMENT_CHOICES = [
        ('CT', 'Computer Engineering'),
        ('ECE', 'Electronics and Communication Engineering'),
        ('MECH', 'Mechanical Engineering'),
        ('CIVIL', 'Civil Engineering'),
        ('EEE', 'Electrical and Electronics Engineering'),
        ('AUTO', 'Automobile Engineering'),
    ]
    SEMESTER_CHOICES = [(i, f'Semester {i}') for i in range(1, 7)]
    GENDER_CHOICES = [
        ('MALE', 'Male'), ('FEMALE', 'Female'), ('OTHER', 'Other'),
    ]

    email = models.EmailField(unique=True)
    user_id = models.CharField(max_length=50, unique=True, null=True, blank=True)
    full_name = models.CharField(max_length=255)
    gender = models.CharField(
        max_length=10, choices=GENDER_CHOICES, null=True, blank=True)
    phone = models.CharField(max_length=15)
    role = models.CharField(max_length=20, choices=ROLE_CHOICES)
    department = models.CharField(
        max_length=10, choices=DEPARTMENT_CHOICES, null=True, blank=True)
    semester = models.IntegerField(
        choices=SEMESTER_CHOICES, null=True, blank=True)
    address = models.TextField(blank=True, null=True)
    bio = models.TextField(blank=True, null=True)
    profile_pic = models.ImageField(
        upload_to='profiles/', blank=True, null=True)
    is_also_class_tutor = models.BooleanField(default=False)
    tutor_semester = models.IntegerField(
        choices=SEMESTER_CHOICES, null=True, blank=True)
    invited_by = models.ForeignKey(
        'self', on_delete=models.SET_NULL, null=True, blank=True,
        related_name='invited_users')
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    must_change_password = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)
    dark_mode = models.BooleanField(default=False)
```

```

objects = UserManager()
USERNAME_FIELD = 'email'
REQUIRED_FIELDS = ['full_name']

def __str__(self):
    return f"{self.full_name} ({self.role})"

@property
def is_principal(self): return self.role == 'PRINCIPAL'
@property
def is_hod(self): return self.role == 'HOD'
@property
def is_class_tutor(self):
    return self.role == 'CLASS_TUTOR' or (
        self.role == 'HOD' and self.is_also_class_tutor)
@property
def is_student(self): return self.role == 'STUDENT'

def can_manage_user(self, target_user):
    if self.is_principal:
        return target_user.role != 'PRINCIPAL'
    elif self.is_hod:
        return (target_user.department == self.department and
            target_user.role in ['CLASS_TUTOR', 'STUDENT'])
    elif self.is_class_tutor:
        return (target_user.role == 'STUDENT' and
            target_user.department == self.department and
            target_user.semester == self.semester)
    return False

class Invite(models.Model):
    created_by = models.ForeignKey(
        User, on_delete=models.CASCADE, related_name='created_invites')
    full_name = models.CharField(max_length=255)
    email = models.EmailField()
    user_id = models.CharField(max_length=50)
    department = models.CharField(
        max_length=10, choices=User.DEPARTMENT_CHOICES)
    semester = models.IntegerField(choices=User.SEMESTER_CHOICES)
    created_at = models.DateTimeField(default=timezone.now)
    is_used = models.BooleanField(default=False)
    used_by = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='used_invites')
    used_at = models.DateTimeField(null=True, blank=True)

    def __str__(self):
        return f"Invite for {self.full_name} ({self.user_id})"

    class Meta:
        ordering = ['-created_at']

class RegistrationRequest(models.Model):
    STATUS_CHOICES = [
        ('PENDING', 'Pending'), ('APPROVED', 'Approved'),
        ('REJECTED', 'Rejected'),
    ]
    user_id = models.CharField(max_length=50)
    full_name = models.CharField(max_length=255)
    email = models.EmailField()
    gender = models.CharField(max_length=10, choices=User.GENDER_CHOICES)
    role = models.CharField(max_length=20, choices=User.ROLE_CHOICES)
    department = models.CharField(
        max_length=10, choices=User.DEPARTMENT_CHOICES)
    semester = models.IntegerField(
        choices=User.SEMESTER_CHOICES, null=True, blank=True)
    password_hash = models.CharField(max_length=255)
    is_also_class_tutor = models.BooleanField(default=False)
    status = models.CharField(
        max_length=20, choices=STATUS_CHOICES, default='PENDING')
    requested_at = models.DateTimeField(default=timezone.now)
    reviewed_at = models.DateTimeField(null=True, blank=True)
    reviewed_by = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='reviewed_registration_requests')

```

```
review_comment = models.TextField(blank=True, null=True)

class Meta:
    ordering = ['-requested_at']

def __str__(self):
    return f"{self.user_id} - {self.full_name} ({self.status})"
```



## 9.2 complaints/models.py

```
from django.db import models
from django.utils import timezone
from accounts.models import User

class Complaint(models.Model):
    STATUS_CHOICES = [
        ('DRAFT', 'Draft'),
        ('PENDING', 'Pending'),
        ('APPROVED_BY_TUTOR', 'Approved by Class Tutor'),
        ('APPROVED_BY_HOD', 'Approved by HOD'),
        ('APPROVED_BY_PRINCIPAL', 'Approved by Principal'),
        ('REJECTED', 'Rejected'),
        ('RESOLVED', 'Resolved'),
    ]
    CATEGORY_CHOICES = [
        ('INFRASTRUCTURE', 'Infrastructure'),
        ('ACADEMICS', 'Academics'),
        ('HOSTEL', 'Hostel'),
        ('CANTEEN', 'Canteen'),
        ('TRANSPORT', 'Transport'),
        ('LIBRARY', 'Library'),
        ('SPORTS', 'Sports'),
        ('OTHER', 'Other'),
    ]

    student = models.ForeignKey(
        User, on_delete=models.CASCADE, related_name='complaints')
    topic = models.CharField(max_length=255)
    category = models.CharField(max_length=50, choices=CATEGORY_CHOICES)
    custom_category = models.CharField(max_length=100, blank=True, null=True)
    description = models.TextField()
    attachment = models.FileField(
        upload_to='complaints/', blank=True, null=True)
    status = models.CharField(
        max_length=30, choices=STATUS_CHOICES, default='DRAFT')
    created_at = models.DateTimeField(default=timezone.now)
    last_modified = models.DateTimeField(auto_now=True)
    submitted_at = models.DateTimeField(null=True, blank=True)
    tutor_status = models.CharField(max_length=20, null=True, blank=True)
    hod_status = models.CharField(max_length=20, null=True, blank=True)
    principal_status = models.CharField(max_length=20, null=True, blank=True)
    reviewed_by_tutor = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='tutor_reviewed_complaints')
    tutor_reviewed_at = models.DateTimeField(null=True, blank=True)
    reviewed_by_hod = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='hod_reviewed_complaints')
    hod_reviewed_at = models.DateTimeField(null=True, blank=True)
    reviewed_by_principal = models.ForeignKey(
        User, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='principal_reviewed_complaints')
    principal_reviewed_at = models.DateTimeField(null=True, blank=True)
    resolved_at = models.DateTimeField(null=True, blank=True)

    def __str__(self):
        return f'{self.topic} - {self.student.full_name}'

    @property
    def display_date(self):
        return self.created_at.strftime('%d/%m/%y')

    @property
    def complaint_display(self):
        return f'{self.topic} {self.display_date}'

    def get_current_action_status(self):
        status_map = {
            'DRAFT': '-',
            'PENDING': 'Pending with Class Tutor',
            'APPROVED_BY_TUTOR': 'Pending with HOD',
```

```

        'APPROVED_BY_HOD': 'Pending with Principal',
        'APPROVED_BY_PRINCIPAL': 'Approved - To be resolved',
        'REJECTED': 'Rejected',
        'RESOLVED': 'Resolved',
    }
    return status_map.get(self.status, '-')

class Meta:
    ordering = ['-created_at']

class ComplaintComment(models.Model):
    complaint = models.ForeignKey(
        Complaint, on_delete=models.CASCADE, related_name='comments')
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    comment = models.TextField()
    created_at = models.DateTimeField(default=timezone.now)
    is_rejection_comment = models.BooleanField(default=False)

    def __str__(self):
        return f"Comment by {self.user.full_name} on {self.complaint.topic}"

class Meta:
    ordering = ['created_at']

class ComplaintStatusNotification(models.Model):
    complaint = models.ForeignKey(
        Complaint, on_delete=models.CASCADE, related_name='notifications')
    user = models.ForeignKey(
        User, on_delete=models.CASCADE, related_name='notifications')
    message = models.TextField()
    created_at = models.DateTimeField(default=timezone.now)
    is_read = models.BooleanField(default=False)

    def __str__(self):
        return f"Notification for {self.user.full_name}"

class Meta:
    ordering = ['-created_at']

```

### 9.3 accounts/views.py (Authentication and Registration)

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, logout, authenticate, update_session_auth_hash
from django.contrib.auth.decorators import login_required
from django.contrib.auth.hashers import make_password
from django.contrib import messages
from django.utils import timezone
from django.http import JsonResponse
from django.urls import reverse
from django.db.models import Q
from .models import User, Invite, RegistrationRequest
from complaints.models import Complaint

def login_view(request):
    if request.user.is_authenticated:
        return redirect('dashboard')
    if request.method == 'POST':
        email = request.POST.get('email')
        password = request.POST.get('password')
        user = authenticate(request, username=email, password=password)
        if user is not None:
            login(request, user)
            if user.must_change_password:
                messages.warning(
                    request,
                    'Please change your temporary password to continue.')
            return redirect('dashboard')
        else:
            messages.error(request, 'Invalid email or password')
    show_register = request.GET.get('show_register') == '1'
    return render(request, 'accounts/login.html',
                  {'show_register': show_register})

def register_view(request):
    if request.user.is_authenticated:
        return redirect('dashboard')
    if request.method == 'GET':
        return redirect(f"{reverse('login')}?show_register=1")
    if request.method == 'POST':
        role = 'STUDENT'
        user_id = (request.POST.get('user_id') or '').strip().upper()
        full_name = (request.POST.get('full_name') or '').strip()
        email = (request.POST.get('email') or '').strip().lower()
        department = (request.POST.get('department') or '').strip().upper()
        semester = (request.POST.get('semester') or '').strip()
        gender = (request.POST.get('gender') or '').strip().upper()
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')

        if password != confirm_password:
            messages.error(request, 'Passwords do not match.')
            return redirect(f"{reverse('login')}?show_register=1")

        invite = Invite.objects.filter(
            user_id__iexact=user_id, is_used=False
        ).order_by('-created_at').first()

        if invite:
            user = User(
                email=invite.email, user_id=invite.user_id.upper(),
                full_name=invite.full_name, gender=gender, phone='',
                role=role, department=invite.department,
                semester=invite.semester, invited_by=invite.created_by
            )
            user.set_password(password)
            user.save()
            invite.is_used = True
            invite.used_by = user
            invite.used_at = timezone.now()
            invite.save(update_fields=['is_used', 'used_by', 'used_at'])
            login(request, user,
```

```

        backend='django.contrib.auth.backends.ModelBackend')
    messages.success(request, 'Registration successful.')
    return redirect('dashboard')

# No invite - create RegistrationRequest for HOD review
RegistrationRequest.objects.create(
    user_id=user_id, full_name=full_name, email=email,
    gender=gender, role=role, department=department,
    semester=int(semester) if semester else None,
    password_hash=make_password(password), status='PENDING'
)
messages.success(
    request,
    'Registration request sent to HOD. Login after approval.')
return redirect('login')
return redirect(f"{reverse('login')}?show_register=1")

```

## 9.4 accounts/views.py (Dashboard View)

```

@login_required
def dashboard_view(request):
    user = request.user

    if user.is_student:
        complaints = Complaint.objects.filter(student=user)
        total_complaints = complaints.exclude(status='DRAFT').count()
        pending_complaints = complaints.filter(status__in=[
            'PENDING', 'APPROVED_BY_TUTOR',
            'APPROVED_BY_HOD', 'APPROVED_BY_PRINCIPAL'
        ]).count()
        pending_list = complaints.filter(status__in=[
            'PENDING', 'APPROVED_BY_TUTOR',
            'APPROVED_BY_HOD', 'APPROVED_BY_PRINCIPAL'
        ]).order_by('-created_at')
        rejected_list = complaints.filter(
            status='REJECTED').order_by('-created_at')
        resolved_list = complaints.filter(
            status='RESOLVED').order_by('-resolved_at')
        context = {
            'total_complaints': total_complaints,
            'pending_complaints': pending_complaints,
            'pending_list': pending_list,
            'rejected_list': rejected_list,
            'resolved_list': resolved_list,
        }
        return render(request, 'complaints/student_dashboard.html', context)

    else:
        # Admin dashboard
        registration_requests = RegistrationRequest.objects.none()
        if user.is_principal:
            complaints = Complaint.objects.filter(
                status__in=['APPROVED_BY_HOD',
                    'APPROVED_BY_PRINCIPAL', 'RESOLVED'])
            pending_complaints = complaints.filter(
                status='APPROVED_BY_HOD')
            resolved_count = complaints.filter(
                status='RESOLVED').count()
            pending_count = complaints.filter(
                status__in=['APPROVED_BY_HOD',
                    'APPROVED_BY_PRINCIPAL']).count()
            managed_users = User.objects.filter(
                role__in=['HOD', 'CLASS_TUTOR', 'STUDENT'])

        elif user.is_hod:
            complaints = _hod_pending_complaints(user)
            pending_complaints = _hod_pending_dashboard_rows(user)
            resolved_count = complaints.filter(
                status='RESOLVED').count()
            pending_count = complaints.filter(status__in=[
                'PENDING', 'APPROVED_BY_TUTOR',
                'APPROVED_BY_HOD', 'APPROVED_BY_PRINCIPAL']).count()
            managed_users = User.objects.filter(
                department=user.department,

```

```

        role__in=['CLASS_TUTOR', 'STUDENT']
    ).order_by('role', 'semester', 'full_name')
    registration_requests = RegistrationRequest.objects.filter(
        department=user.department,
        status='PENDING').order_by('-requested_at')

else: # Class Tutor
    complaints = Complaint.objects.filter(
        student__department=user.department,
        student__semester=user.semester,
        status__in=['PENDING', 'APPROVED_BY_TUTOR',
                    'APPROVED_BY_HOD', 'APPROVED_BY_PRINCIPAL',
                    'RESOLVED'])
    pending_complaints = complaints.filter(status='PENDING')
    resolved_count = complaints.filter(
        status='RESOLVED').count()
    pending_count = complaints.filter(status__in=[
        'PENDING', 'APPROVED_BY_TUTOR']).count()
    managed_users = User.objects.filter(
        department=user.department,
        semester=user.semester,
        role='STUDENT').order_by('full_name')

    invites = Invite.objects.filter(
        created_by=user, is_used=False).order_by('-created_at')

    context = {
        'complaints': complaints,
        'pending_complaints': pending_complaints,
        'pending_count': pending_count,
        'resolved_count': resolved_count,
        'managed_users': managed_users,
        'invites': invites,
        'registration_requests': registration_requests,
    }
return render(request, 'complaints/admin_dashboard.html', context)

```

## 9.5 complaints/views.py (Complaint Management)

```
@login_required
def manage_complaints_view(request):
    user = request.user
    if user.is_student:
        if request.method == 'POST':
            action = request.POST.get('action', 'draft')
            topic = (request.POST.get('topic') or '').strip()
            category = request.POST.get('category')
            description = request.POST.get('description')
            complaint = Complaint(
                student=user, topic=topic,
                category=category or 'OTHER',
                description=description or '',
                attachment=request.FILES.get('attachment'),
            )
            if action == 'submit':
                complaint.status = 'PENDING'
                complaint.submitted_at = timezone.now()
                complaint.tutor_status = 'Pending'
                complaint.save()
                messages.success(request, 'Complaint saved successfully.')
                return redirect('manage_complaints')

        drafts = Complaint.objects.filter(student=user, status='DRAFT')
        pending = Complaint.objects.filter(student=user, status__in=[
            'PENDING', 'APPROVED_BY_TUTOR',
            'APPROVED_BY_HOD', 'APPROVED_BY_PRINCIPAL'])
        resolved = Complaint.objects.filter(
            student=user, status='RESOLVED')
        rejected = Complaint.objects.filter(
            student=user, status='REJECTED')
        context = {
            'drafts': drafts, 'pending': pending,
            'resolved': resolved, 'rejected': rejected,
        }
        return render(
            request, 'complaints/student_manage_complaints.html', context)
    else:
        # Admin: group complaints by category
        if user.is_principal:
            complaints = Complaint.objects.filter(
                status__in=['APPROVED_BY_HOD',
                    'APPROVED_BY_PRINCIPAL', 'RESOLVED'])
        elif user.is_hod:
            complaints = Complaint.objects.filter(
                student__department=user.department,
                status__in=['APPROVED_BY_TUTOR', 'APPROVED_BY_HOD',
                    'APPROVED_BY_PRINCIPAL', 'RESOLVED'])
        else:
            complaints = Complaint.objects.filter(
                student__department=user.department,
                student__semester=user.semester,
                status__in=['PENDING', 'APPROVED_BY_TUTOR', 'RESOLVED'])
        categories = {}
        for c in complaints:
            categories.setdefault(c.category, []).append(c)
        return render(request, 'complaints/admin_manage_complaints.html',
            {'categories': categories})
```

## 9.6 complaints/views.py (Approval and Rejection Actions)

```
@login_required
def manage_complaint_action(request, complaint_id):
    if request.user.is_student:
        return redirect('dashboard')
    complaint = get_object_or_404(Complaint, id=complaint_id)
    user = request.user

    if request.method == 'POST':
        action = request.POST.get('action')
```

```

comment_text = (request.POST.get('comment') or '').strip()

if not comment_text:
    messages.error(request, 'Comment is required.')
    return redirect('manage_complaint_action',
                    complaint_id=complaint_id)

if action == 'approve':
    if user.is_class_tutor:
        complaint.status = 'APPROVED_BY_TUTOR'
        complaint.reviewed_by_tutor = user
        complaint.tutor_reviewed_at = timezone.now()
        complaint.tutor_status = 'Approved'
        complaint.hod_status = 'Pending'
        notification_msg = (
            f'Your complaint "{complaint.topic}" has been '
            f'approved by Class Tutor and is pending with HOD.')
    elif user.is_hod:
        complaint.status = 'APPROVED_BY_HOD'
        complaint.reviewed_by_hod = user
        complaint.hod_reviewed_at = timezone.now()
        complaint.hod_status = 'Approved'
        complaint.principal_status = 'Pending'
        notification_msg = (
            f'Your complaint "{complaint.topic}" has been '
            f'approved by HOD and is pending with Principal.')
    elif user.is_principal:
        complaint.status = 'APPROVED_BY_PRINCIPAL'
        complaint.reviewed_by_principal = user
        complaint.principal_reviewed_at = timezone.now()
        complaint.principal_status = 'Approved'
        notification_msg = (
            f'Your complaint "{complaint.topic}" has been '
            f'approved by Principal and will be resolved soon.')
    elif action == 'reject':
        complaint.status = 'REJECTED'
        notification_msg = (
            f'Your complaint "{complaint.topic}" has been rejected. '
            f'Check comments for details.')

    complaint.save()
    ComplaintComment.objects.create(
        complaint=complaint, user=user,
        comment=comment_text,
        is_rejection_comment=(action == 'reject'))
    ComplaintStatusNotification.objects.create(
        complaint=complaint,
        user=complaint.student,
        message=notification_msg)
    messages.success(request, 'Action recorded successfully.')
    return redirect('dashboard')

return render(request, 'complaints/manage_complaint_action.html',
              {'complaint': complaint})

@login_required
def mark_resolved(request, complaint_id):
    if not request.user.is_principal:
        messages.error(request, 'Only Principal can resolve complaints.')
        return redirect('dashboard')
    complaint = get_object_or_404(
        Complaint, id=complaint_id, status='APPROVED_BY_PRINCIPAL')
    if request.method == 'POST':
        complaint.status = 'RESOLVED'
        complaint.resolved_at = timezone.now()
        complaint.save()
        ComplaintStatusNotification.objects.create(
            complaint=complaint,
            user=complaint.student,
            message=f'Your complaint "{complaint.topic}" has been resolved.')
        messages.success(request, 'Complaint marked as resolved.')
        return redirect('dashboard')
    return render(request, 'complaints/confirm_resolve.html',
                  {'complaint': complaint})

```

## 9.7 accounts/views.py (User Account Creation)

```
@login_required
def create_user_account(request):
    if request.method == 'POST':
        user = request.user
        role = (request.POST.get('role') or '').strip().upper()
        department = (request.POST.get('department') or '').strip().upper()
        semester = request.POST.get('semester') or None
        is_hod_also_tutor = request.POST.get('is_also_class_tutor') == 'on'
        hod_tutor_semester = request.POST.get('hod_tutor_semester') or None
        temporary_password = request.POST.get('temporary_password') or ''
        full_name = (request.POST.get('full_name') or '').strip()
        email = (request.POST.get('email') or '').strip().lower()
        user_id = (request.POST.get('user_id') or '').strip().upper()

        if role == 'HOD' and is_hod_also_tutor:
            semester = hod_tutor_semester

        # Permission check
        if user.is_principal:
            allowed_roles = ['HOD', 'CLASS_TUTOR', 'STUDENT']
        elif user.is_hod:
            allowed_roles = ['CLASS_TUTOR', 'STUDENT']
        elif user.is_class_tutor:
            allowed_roles = ['STUDENT']
        else:
            return JsonResponse({'success': False, 'error': 'Denied'})

        if role not in allowed_roles:
            return JsonResponse(
                {'success': False, 'error': 'Not allowed to create this role'})

        if role == 'STUDENT':
            # Create invite for student self-registration
            invite = Invite.objects.create(
                created_by=user, full_name=full_name, email=email,
                user_id=user_id, department=department,
                semester=int(semester))
            return JsonResponse(
                {'success': True, 'user_id': invite.user_id,
                 'direct_account': False})

        # Direct account creation for HOD and Class Tutor
        new_user = User(
            role=role, email=email, user_id=user_id,
            full_name=full_name, phone='', department=department,
            semester=int(semester) if role == 'CLASS_TUTOR' else None,
            is_also_class_tutor=(role == 'HOD' and is_hod_also_tutor),
            tutor_semester=(int(semester)
                            if role == 'HOD' and is_hod_also_tutor
                            else None),
            is_active=True, must_change_password=True,
        )
        new_user.set_password(temporary_password)
        new_user.save()
        return JsonResponse(
            {'success': True, 'user_id': new_user.user_id,
             'direct_account': True})
    return JsonResponse({'success': False})
```

## 9.8 accounts/views.py (Semester Increment)

```
@login_required
def increment_semester(request):
    if not request.user.is_principal:
        messages.error(request, 'Only Principal can increment semesters.')
        return redirect('dashboard')

    if request.method == 'POST':
        # Remove all Semester 6 students (graduated)
        deleted_count, _ = User.objects.filter(
```



```

        role='STUDENT', semester=6).delete()

    # Increment remaining students
    students = User.objects.filter(
        role='STUDENT', is_active=True).exclude(semester=6)
    incremented_count = 0
    skipped_count = 0
    for student in students:
        if student.semester is None:
            skipped_count += 1
            continue
        student.semester += 1
        student.save()
        incremented_count += 1

    messages.success(
        request,
        f'Semesters incremented: {incremented_count} promoted, '
        f'{deleted_count} Semester 6 accounts deleted, '
        f'{skipped_count} skipped.')
    return redirect('dashboard')

return render(request, 'accounts/confirm_semester_increment.html')

```

## 9.9 config/urls.py

```

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('accounts.urls')),
    path('', include('complaints.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

## 9.10 accounts/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.redirect_root, name='root'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('register/', views.register_view, name='register'),
    path('invite-lookup/', views.invite_registration_lookup,
         name='invite_registration_lookup'),
    path('dashboard/', views.dashboard_view, name='dashboard'),
    path('settings/', views.settings_view, name='settings'),
    path('users/create/', views.create_user_account,
         name='create_user_account'),
    path('users/<int:user_id>/edit/', views.edit_user_view,
         name='edit_user'),
    path('users/<int:user_id>/delete/', views.delete_user,
         name='delete_user'),
    path('invites/<int:invite_id>/delete/', views.delete_invite,
         name='delete_invite'),
    path('registration-requests/<int:request_id>/review/',
         views.review_registration_request,
         name='review_registration_request'),
    path('semester-increment/', views.increment_semester,
         name='semester_increment'),
    path('change-temp-password/', views.change_temporary_password,
         name='change_temporary_password'),
]

```

## 9.11 complaints/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('complaints/manage/', views.manage_complaints_view,
         name='manage_complaints'),
    path('complaints/view/<int:complaint_id>/', views.view_complaint,
         name='view_complaint'),
    path('complaints/edit/<int:complaint_id>/', views.edit_complaint,
         name='edit_complaint'),
    path('complaints/delete/<int:complaint_id>/', views.delete_complaint,
         name='delete_complaint'),
    path('complaints/action/<int:complaint_id>/',
         views.manage_complaint_action, name='manage_complaint_action'),
    path('complaints/resolve/<int:complaint_id>/', views.mark_resolved,
         name='mark_resolved'),
]
```

## 9.12 templates/accounts/login.html

```
{% load static %}
<!doctype html>
<html lang="en" id="loginHtml">
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Login - SCRS</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/
    bootstrap.min.css" rel="stylesheet"/>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/
    bootstrap-icons@1.11.1/font/bootstrap-icons.css"/>
  <link rel="stylesheet" href="{% static 'css/login.css' %}"/>
</head>
<body>
  <div class="sidebar">
    <div class="sidebar-header"></div>
    <nav class="nav flex-column">
      <a class="nav-link active">
        <i class="bi bi-box-arrow-in-right"></i> Login
      </a>
      <a class="nav-link disabled">
        <i class="bi bi-speedometer2"></i> Dashboard
      </a>
    </nav>
  </div>

  <div class="main-content">
    <div class="title-bar">
      <h6 class="mb-0">Login</h6>
      <i class="bi bi-moon-fill theme-toggle" id="themeToggle"></i>
    </div>
    <div class="login-container">
      <div class="login-card">
        <div class="login-header">
          <h2>GPTCK</h2>
          <p>Complaint Management System</p>
        </div>
        {% if messages %}
          {% for message in messages %}
            <div class="alert alert-{{ message.tags }} alert-dismissible
              fade show" role="alert">
              {{ message }}
              <button type="button" class="btn-close"
                data-bs-dismiss="alert"></button>
            </div>
          {% endfor %}
        {% endif %}
        <form method="post">
          {% csrf_token %}
          <div class="mb-3">
            <label for="email" class="form-label">Email</label>
            <input type="email" class="form-control" id="email"
              name="email" required/>
          </div>
          <div class="mb-4">
            <label for="password" class="form-label">Password</label>
            <div class="password-wrapper">
              <input type="password" class="form-control" id="password"
                name="password" required style="padding-right: 40px"/>
              <i class="bi bi-eye-slash password-toggle"
                id="passwordToggle"></i>
            </div>
          </div>
          <button type="submit" class="btn btn-primary w-100">
            Login
          </button>
        </form>
        <div class="register-link">
          <a href="#" data-bs-toggle="modal"
            data-bs-target="#registerModal">
            New student? User Registration
          </a>
        </div>
      </div>
    </div>
  </div>
```

```

        </div>
        <div class="footer-text">
            &copy; 2026 Government Polytechnic College Kalamassery
        </div>
    </div>
</div>
<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/
    bootstrap.bundle.min.js"></script>
<script>
    // Theme toggle using localStorage
    const html = document.getElementById('loginHtml');
    const themeToggle = document.getElementById('themeToggle');
    const savedTheme = localStorage.getItem('theme') || 'light';
    html.setAttribute('data-theme', savedTheme);
    html.setAttribute('data-bs-theme', savedTheme);
    themeToggle.addEventListener('click', function() {
        const currentTheme = html.getAttribute('data-theme');
        const newTheme = currentTheme === 'dark' ? 'light' : 'dark';
        html.setAttribute('data-theme', newTheme);
        html.setAttribute('data-bs-theme', newTheme);
        localStorage.setItem('theme', newTheme);
    });
    // Password visibility toggle
    document.getElementById('passwordToggle').addEventListener(
        'click', function() {
            const input = document.getElementById('password');
            input.type = input.type === 'password' ? 'text' : 'password';
        });
    {% if show_register %}
    new bootstrap.Modal(
        document.getElementById('registerModal')).show();
    {% endif %}
</script>
</body>
</html>

```

## 9.13 templates/complaints/student\_manage\_complaints.html

```
{% extends 'shared/base.html' %}
{% block page_title %}Manage Complaints{% endblock %}
{% block content %}

<!-- Drafts Table -->
<div class="table-container">
  <h5 class="mb-3">Drafts</h5>
  <div class="table-responsive">
    <table class="table table-hover">
      <thead>
        <tr>
          <th>Complaint</th>
          <th>Last Modified</th>
          <th>Options</th>
        </tr>
      </thead>
      <tbody>
        {% for complaint in drafts %}
        <tr>
          <td>{{ complaint.complaint_display }}</td>
          <td>{{ complaint.last_modified|date:"d/m/y H:i" }}</td>
          <td>
            <a href="{% url 'view_complaint' complaint.id %}"
              class="btn btn-sm btn-outline-primary">View</a>
            <a href="{% url 'edit_complaint' complaint.id %}"
              class="btn btn-sm btn-outline-warning">Edit</a>
            <form method="post"
              action="{% url 'delete_complaint' complaint.id %}"
              style="display: inline;">
              {% csrf_token %}
              <button type="submit" class="btn btn-sm btn-outline-danger"
                onclick="return confirm('Delete this draft?')">
                Delete
              </button>
            </form>
          </td>
        </tr>
        {% empty %}
        <tr>
          <td colspan="3" class="text-center text-muted">No drafts</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</div>

<!-- Pending Complaints Table -->
<div class="table-container">
  <h5 class="mb-3">Pending</h5>
  <div class="table-responsive">
    <table class="table table-hover">
      <thead>
        <tr>
          <th>Complaint</th>
          <th>Current Stage</th>
          <th>Options</th>
        </tr>
      </thead>
      <tbody>
        {% for complaint in pending %}
        <tr>
          <td>{{ complaint.complaint_display }}</td>
          <td>
            <span class="badge bg-warning text-dark">
              {{ complaint.get_current_action_status }}
            </span>
          </td>
          <td>
            <a href="{% url 'view_complaint' complaint.id %}"
              class="btn btn-sm btn-outline-primary">View</a>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
```

```

        </tr>
        {% empty %}
        <tr>
            <td colspan="3" class="text-center text-muted">
                No pending complaints
            </td>
        </tr>
        {% endfor %}
    </tbody>
</table>
</div>
</div>

<!-- Resolved Complaints Table -->
<div class="table-container">
    <h5 class="mb-3">Resolved</h5>
    <table class="table table-hover">
        <thead>
            <tr><th>Complaint</th><th>Resolved On</th><th>Options</th></tr>
        </thead>
        <tbody>
            {% for complaint in resolved %}
            <tr>
                <td>{{ complaint.complaint_display }}</td>
                <td>{{ complaint.resolved_at|date:"d/m/y" }}</td>
                <td>
                    <a href="{% url 'view_complaint' complaint.id %}"
                        class="btn btn-sm btn-outline-success">View</a>
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="3" class="text-center text-muted">
                    No resolved complaints
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}

```

## 9.14 setup\_principal.py

```

import os
import django

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup()

from accounts.models import User

email = 'principal@gptck.ac.in'
password = 'Principal@gptck_1951'
user_id = 'PRINCIPAL001'
full_name = 'Principal'

if User.objects.filter(email=email).exists():
    user = User.objects.get(email=email)
    user.set_password(password)
    user.user_id = user_id
    user.full_name = full_name
    user.role = 'PRINCIPAL'
    user.is_staff = True
    user.is_superuser = True
    user.save()
    print(f'Principal account updated: {email}')
else:
    User.objects.create_superuser(
        email=email,
        password=password,
        full_name=full_name,
        user_id=user_id,
        phone='0000000000',
    )

```

```
)  
    print(f'Principal account created: {email}')
```

print('Default credentials:')  
print(f' Email : {email}')

print(f' Password: {password}')

print(f' User ID : {user\_id}')

## **10. SYSTEM TESTING AND IMPLEMENTATION**

### **10.1 Testing Objectives**

The primary objectives of system testing were to verify that all user roles function as specified, the complaint workflow transitions occur correctly, access control prevents unauthorised actions, and edge cases such as duplicate registrations and invalid form submissions are handled gracefully.

### **10.2 Testing Methodologies**

- **Unit Testing:** Individual views and model methods were tested in isolation. Django's built-in test client (`django.test.Client`) was used to send simulated HTTP requests and verify response codes, redirect targets, and database state.
- **Integration Testing:** The interaction between the accounts and complaints applications was verified. Key integration points include the student-complaint foreign key, the role-based filtering of complaints in dashboard and manage views, and the notification creation on status change.
- **Functional Testing:** All specified user stories were tested end-to-end. For example: a student registers using an invite, logs in, creates a draft complaint, submits it, the Class Tutor approves it, the HOD approves it, the Principal approves and resolves it, and the student sees the resolved status.
- **Negative Testing:** Attempts were made to access views beyond a user's role (e.g., a Student accessing the complaint action URL), to submit forms with missing required fields, and to reuse a consumed invite. All negative cases were verified to produce appropriate error messages.
- **Security Testing:** CSRF token validation was confirmed on all POST forms. Session authentication was verified by attempting to access login-protected views without a session. Password hashing was confirmed by inspecting the database — no plaintext passwords were stored.
- **Usability Testing:** The system was reviewed for clarity of navigation, consistency of feedback messages, and mobile responsiveness on a 375px-wide viewport (iPhone SE equivalent).

### **10.3 Implementation Steps**

1. Install dependencies: `pip install -r requirements.txt`
2. Apply database migrations: `python manage.py migrate`
3. Create Principal account: `python setup_principal.py`
4. Collect static files (production): `python manage.py collectstatic`
5. Start development server: `python manage.py runserver`
6. Open browser at `http://127.0.0.1:8000/` and log in as Principal.
7. Create HOD accounts for each department from the Principal dashboard.
8. Each HOD logs in, changes their temporary password, and creates Class Tutor accounts.
9. Class Tutors log in, change their temporary passwords, and create invites for students.
10. Students register using their invite IDs and begin using the system.



## **11. SYSTEM MAINTENANCE**

System maintenance refers to the ongoing activities required to keep the Student Complaint Registration System operational, up-to-date, and aligned with the evolving needs of the institution. Four categories of maintenance are applicable to this system.

### **11.1 Corrective Maintenance**

Corrective maintenance addresses bugs, errors, and unexpected behaviours discovered after deployment. The following issues represent typical corrective maintenance scenarios for SCRS:

- Status transition errors: If a complaint status transition occurs incorrectly (e.g., a complaint jumping from PENDING to APPROVED\_BY\_PRINCIPAL without passing through the HOD stage), the transition logic in complaints/views.py must be reviewed and corrected.
- Access control gaps: If a user is able to access a view or perform an action outside their role due to a missing permission check, the affected view must be patched.
- Registration edge cases: If duplicate user\_id or email registrations are possible through a race condition (two simultaneous registrations with the same ID), a database-level UNIQUE constraint (already present on the model) and server-side validation must be verified.
- Notification not created: If a status change does not produce a ComplaintStatusNotification, the notification creation logic in the affected view must be checked and corrected.
- File upload failures: If file attachments fail to save due to missing media directory or misconfigured MEDIA\_ROOT, the settings.py and directory permissions must be corrected.

•

### **11.2 Adaptive Maintenance**

Adaptive maintenance updates the system to remain compatible with changes in the external environment:

- Django version upgrades: When Django releases security or feature updates within the 4.x series, the project should be updated by running pip install --upgrade Django==4.x.x and re-running migrations if any schema changes are required.
- Department additions: If the college adds a new engineering department, the DEPARTMENT\_CHOICES list in accounts/models.py must be updated and a migration generated with python manage.py makemigrations.
- Semester structure changes: If the college adopts a credit-based system with more than 6 semesters, the SEMESTER\_CHOICES list and the semester increment logic must be updated.
- Bootstrap upgrades: If Bootstrap is upgraded to version 6.x, template files must be reviewed for deprecated classes and updated accordingly.

### **11.3 Perfective Maintenance**

Perfective maintenance improves the system beyond its current capabilities:

- Email notifications: Adding Django's email backend (SMTP or a transactional service) to send email alerts when complaints are approved, rejected, or resolved.
- Search and filtering: Adding a search bar and date-range filter to complaint lists for large complaint volumes.
- Pagination: Adding Django's Paginator to complaint and user lists to improve performance when the database grows large.

- Analytics dashboard: Adding complaint statistics (resolution rates, average time to resolve, complaints by department) to the Principal dashboard using Chart.js or a server-side charting library.
- UI improvements: Refining the responsive layout for smaller mobile screens, improving accessibility for screen readers, and adding keyboard navigation support.

#### **11.4 Preventive Maintenance**

Preventive maintenance reduces the probability of future failures:

- Database backups: Regular automated backups of the SQLite file (or PostgreSQL database in production) using cron jobs or cloud backup services.
- Dependency audits: Periodic review of all installed packages for known security vulnerabilities using pip audit or GitHub Dependabot.
- Inactive account management: Periodic deactivation of student accounts that are no longer enrolled, using the `is_active` flag on the User model.
- Log monitoring: Enabling Django's logging framework to capture errors and warnings to a log file, and reviewing logs periodically for unusual activity.
- SSL certificate renewal: Ensuring that the HTTPS certificate (if using Let's Encrypt) is renewed before expiry using Certbot's auto-renewal functionality.

## **12. FUTURE SCOPE**

The current version of the Student Complaint Registration System provides a solid foundation for institutional grievance management. Several enhancements have been identified that would significantly extend the system's utility and impact.

### **12.1 Email and SMS Notifications**

The most immediately valuable enhancement would be the addition of real-time email notifications. Currently, students must log in to the system to check notification status. By integrating Django's email backend with a transactional email service such as SendGrid or the college's own SMTP server, the system could send automatic email alerts whenever a complaint is approved, rejected, or resolved. Similarly, SMS notifications via a service like Twilio would benefit students who may not have regular access to email. This would dramatically reduce the time between a status change and student awareness.

### **12.2 Mobile Application**

A dedicated mobile application for Android and iOS would significantly improve accessibility for students who primarily use smartphones. The application could be built using a cross-platform framework such as Flutter or React Native. The existing Django backend could be extended with a REST API layer using Django REST Framework (DRF), exposing endpoints for authentication, complaint CRUD operations, and notification retrieval. Push notifications via Firebase Cloud Messaging (FCM) could replace the current in-app notification model, ensuring students are alerted even when the app is backgrounded.

### **12.3 Analytics and Reporting Dashboard**

An analytics module would provide the Principal and HODs with actionable insights into institutional grievance patterns. Metrics of interest include: the total number of complaints per department, per semester, and per category; the average time taken for a complaint to move through each stage of the approval chain; the rejection rate at each level; and the proportion of complaints that are ultimately resolved versus rejected. These metrics could be displayed as interactive charts using Chart.js or a Python visualisation library (Matplotlib or Plotly) embedded in the dashboard. Periodic PDF reports could be generated and emailed to the Principal automatically.

### **12.4 Service Level Agreement (SLA) and Auto-Escalation**

A significant weakness of any manual approval workflow is the risk of complaints stalling at a particular level. An SLA mechanism would automatically flag complaints that have not been acted upon within a defined number of working days (e.g., 3 days at the Tutor level, 5 days at the HOD level). If the SLA deadline is missed, the system would automatically escalate the complaint to the next authority level and notify both the student and the bypassed authority. This would prevent institutional inertia from blocking legitimate grievance resolution.

### **12.5 Anonymous Complaint Option**

Some grievances, particularly those involving faculty misconduct, are sensitive enough that students may fear retaliation. An anonymous complaint option, where the student's identity is concealed from lower-level reviewers but visible only to the Principal, could encourage students to report serious

issues they might otherwise suppress. The implementation would require a flag on the Complaint model and conditional display logic in admin templates.

## **12.6 Multi-Institution Support**

The system's architecture is well-suited for extension to a multi-institution platform serving all polytechnic colleges under the Department of Technical Education, Kerala. A super-admin role above the Principal level could be introduced to manage multiple institution instances. Each institution would have its own set of departments, user hierarchy, and complaint workflows, all within a single centralised deployment. This would allow the Department of Technical Education to monitor grievance patterns across all polytechnic colleges and identify systemic issues requiring policy-level intervention.

## **12.7 AI-Based Complaint Classification**

Natural Language Processing (NLP) could be applied to automatically suggest the most appropriate complaint category based on the student's free-text description. A simple text classification model trained on historical complaint data could be integrated with the complaint creation form using a lightweight Python NLP library. This would reduce miscategorisation and improve the consistency of complaint routing, making it easier for administrators to prioritise their workloads.

### **13. CONCLUSION**

The Student Complaint Registration System (SCRS) is a well-structured, role-based web application that addresses the critical need for a formal, transparent, and accountable grievance management mechanism at Government Polytechnic College, Kalamassery. The system replaces ad-hoc, paper-based, and verbal complaint handling with a structured digital workflow that enforces accountability at every level of the academic hierarchy.

The system's core strength lies in its multi-level approval workflow, which mirrors the real institutional hierarchy of the college: Student, Class Tutor, Head of Department, and Principal. This hierarchy is enforced through role-based access control implemented in Django's custom user model. Every action in the system — whether a complaint submission, an approval, or a rejection — is timestamped, attributed to a specific user, and permanently recorded in the database. This provides a complete and auditable trail for every complaint from creation to resolution.

The dual registration mechanism — invite-based for controlled and immediate account creation, and request-based for student-initiated registration with HOD oversight — strikes the right balance between institutional control and practical accessibility. The temporary-password flow for authority-created accounts ensures that no system account remains accessible with a shared credential.

The use of Python Django 4.2 as the backend framework provides a mature, secure, and maintainable foundation. Django's ORM abstracts database interactions, its built-in CSRF protection secures all form submissions, and its session-based authentication reliably identifies users across requests. Bootstrap 5 provides a clean, responsive, and accessible frontend that works well on both desktop computers and mobile devices — important for a student population that increasingly relies on smartphones.

The mandatory comment requirement at every approval or rejection stage is a particularly important design decision. It ensures that authorities cannot dismiss complaints silently — every decision must be justified in writing. This documentation protects both the student (who receives a reason for rejection) and the institution (which has a record of why each complaint was handled as it was).

The semester increment tool, the complaint attachment feature, the in-app notification system, and the profile management capabilities collectively make SCRS a comprehensive institutional tool rather than a minimal complaint tracker. The system is ready for immediate deployment and is designed to scale to the full student body of Government Polytechnic College, Kalamassery, across all six engineering departments.

With future enhancements such as email notifications, a mobile application, analytics dashboards, SLA-driven auto-escalation, and multi-institution support, SCRS has the potential to evolve into a comprehensive grievance management platform for the broader polytechnic college ecosystem in Kerala. The project demonstrates that open-source technologies, applied thoughtfully, can deliver a professional-grade institutional management system at minimal cost.

## 14. REFERENCES

1. Django Software Foundation. (2024). Django 4.2 Documentation. Retrieved from <https://docs.djangoproject.com/en/4.2/>
2. Holovaty, A., & Kaplan-Moss, J. (2008). The Definitive Guide to Django: Web Development Done Right (2nd ed.). Apress.
3. Vincent, W. S. (2022). Django for Beginners: Build Websites with Python and Django (4th ed.). LearnDjango.com LLC.
4. Bootstrap Team. (2024). Bootstrap 5.3 Documentation. Retrieved from <https://getbootstrap.com/docs/5.3/>
5. Percival, H. J. W. (2017). Test-Driven Development with Python: Obey the Testing Goat (2nd ed.). O'Reilly Media.
6. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
7. Mozilla Developer Network. (2024). Web technology documentation. Retrieved from <https://developer.mozilla.org/>
8. Python Software Foundation. (2024). Python 3 Documentation. Retrieved from <https://docs.python.org/3/>
9. OWASP Foundation. (2024). OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org/www-project-top-ten/>
10. Brown, P. (2021). Web Application Security: Exploitation and Countermeasures. O'Reilly Media.
11. Django Software Foundation. (2024). Writing and running tests in Django. Retrieved from <https://docs.djangoproject.com/en/4.2/topics/testing/>
12. Bootstrap Icons. (2024). Bootstrap Icons v1.11 Library. Retrieved from <https://icons.getbootstrap.com/>
13. Government of Kerala – Department of Technical Education. (2024). Polytechnic College Curriculum Guidelines. Thiruvananthapuram: DTE Kerala.
14. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
15. Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.